Structures that Control Flow

Section 1

Chapter 3

An Introduction to Programming Using Python
David I. Schneider

The `bool` Data Type

- A primitive data type with 2 possible **values**:
  - **True** or **False**
  - data type: **`bool`**.
  - Examples:
    - 2 > 5
    - 'e' in 'Hello'
  - What do these lines display?

```
x = 2
y = 3
var = x < y
print(var)
```

True

```
x = 5
print((3 + x) < 7)
```

False

Relational and Logical Operators

- A *condition* is an expression
  - Involving relational operators (such as < and >=)
  - Logical operators (such as and, or, and not)
  - Evaluates to either **True** or **False**
  - Example: x+y > 2 and z <= 0
- Conditions used to make decisions
  - Control loops
  - Choose between options

Relational and Logical Operators

- Rules for expressions with non-Boolean data types
  - int: 0 →False; All other values →True
  - float: 0.0 →False; All other values →True
  - list: [] (i.e. empty list) →False; All other values →True
  - str: '' (i.e. empty string) →False; All other values →True

Comparison Operators

- Basic comparison operators: <, <=, ==, !=, >=, >
  - a = 4, b=12
  - a < b
  - True
  - a == b
  - False
  - a+b != 9+9
  - True

Membership Operator

- *in*: tests for membership, returns True or False
- *not in*: tests for non-membership, returns True or False
- L = [1, [2,3], 'ab', -23] S = 'Python is great!'
  - [2,3] in L → True
  - 2 in L → False
  - 2 not in L → True
  - “on is” in S → True
  - “eat” not in S → False

Relational Operators

Python Notation	Numeric Meaning	String Meaning
==	equal to	identical to
!=	not equal to	different from
<	less than	precedes lexicographically
>	greater than	follows lexicographically
<=	less than or equal to	precedes lexicographically or is identical to
>=	greater than or equal to	follows lexicographically or is identical to
in		substring of
not in		not a substring of

Table 3.3 Relational operators.

Relational Operators

- Relational operator *less than* (<) can be applied to
  - Numbers
  - Strings
  - Other objects
- For strings, the ASCII table determines order of characters

ASCII Values

- ASCII values determine order used to compare strings with relational operators.
- Associated with keyboard letters, characters, numerals
  - ASCII values are numbers ranging from 32 to 126.
- A few ASCII values.

32 (space)	48 0	66 B	122 z
33 !	49 1	90 Z	123 {
34 “	57 9	97 a	125 }
35 #	65 A	98 b	126 ~

ASCII Values

- The ASCII standard also assigns characters to some numbers above 126.
- A few of the higher ASCII values.

162 ¢	177 ±	181 µ	190 ¾
169 ©	178 ²	188 ¼	247 ÷
176 °	179 ³	189 ½	248 ø

- Functions **`chr(n)`** and **`ord(str)`** access ASCII values

Relational Operators

- Some rules
  - An int *can* be compared to a float.
  - Otherwise, values of different types *cannot* be compared
  - Relational operators can be applied to lists or tuples

```
[3, 5] < [3, 7]
[3, 5] < [3, 5, 6]
[3, 5, 7] < [3, 7, 2]
[7, "three", 5] < [7, "two", 2]
```

Comparing Strings

- Use ASCII values of characters.
  - ord(x) → Returns integer ASCII value of a character x
  - chr(n) → Returns the character corresponding to integer n
  - Example: ord('a') → 97; chr(60) → '<'

Evaluate the following:

- 'cat' < 'dog'
  - True
- 'cat' < 'car'
  - False
- 'cat' < 'carry'
  - False

Relational Operators

- Example 1: Determine whether following conditions evaluate to *True* or *False*

(a) $1 <= 1$
(b) $1 < 1$
(c) "car" < "cat"
(d) "Dog" < "dog"
(e) "fun" in "refunded"

Relational Operators

- Example 2: Determine whether following conditions evaluate to *True* or *False*

(a) (a + b) < (2 * a)

(b) (len(c) − b) == (a/2)

(c) c < ("good" + d)

Logical Operators

- Enables combining multiple relational operators
- Logical operators are the reserved words **`and`**, **`or`**, and **`not`**
- Conditions that use these operators are called **compound** conditions

Logical Operators

- Given: *cond1* and *cond2* are conditions
  - **`cond1 and cond2`** true only if both conditions are true
  - **`cond1 or cond2`** true if either or both conditions are true
  - **`not cond1`** is false if the condition is true, true if the condition is false

Logical Operators

- Example 6: Given n = 4, answ = "Y"
  Determine expressions = *True* or *False*

(a) (2 < n) and (n < 6)
(b) (2 < n) or (n == 6)
(c) not (n < 6)
(d) (answ == "Y") or (answ == "y")
(e) (answ == "Y") and (answ == "y")
(f) not (answ == "y")
(g) ((2 < n) and (n == 5 + 1)) or (answ == "No")
(h) ((n == 2) and (n == 7)) or (answ == "Y")
(i) (n == 2) and ((n == 7) or (answ == "Y"))

Short-Circuit Evaluation

- Consider the condition **`cond1 and cond2`**
  - If Python evaluates **`cond1`** as false, it does not bother to check **`cond2`**
- Similarly with **`cond1 or cond2`**
  - If Python finds **`cond1`** true, it does not bother to check further
- Think why this feature helps for

```
(number != 0) and (m == (n / number))
```

Logical Operators

- short circuit evaluation → return operand that determined result, rather than a Boolean value (unless operand itself is Boolean)
  - 5 and 2 → 2
  - 0 and 2 → 0
  - 0 or 7 or 8 → 7
  - 6 or 6 or 9 → 6
  - not 6 → False
  - not 0 → True
  - not '0' → False

Methods that Return Boolean Values

- Given: strings **`str1`** and **`str2`**
  - **`str1.startswith(str2)`**
  - **`str1.endswith(str2)`**
- For determining the type of an item
  - **`isinstance(item, dataType)`**

Methods that Return Boolean Values

- Methods that return either True or False.

Method	Returns True when
str1.isdigit()	all of *str1*'s characters are digits
str1.isalpha()	all of *str1*'s characters are letters of the alphabet
str1.isalnum()	all of *str1*'s characters are letters of the alphabet or digits
str1.islower()	*str1* has at least 1 alphabetic character and all of its alphabetic characters are lowercase
str1.isupper()	*str1* has at least 1 alphabetic character and all of its alphabetic characters are uppercase
str1.isspace()	*str1* contains only whitespace characters

Table 3.4 Methods that return either True or False.

End

Section 1

Chapter 3

An Introduction to Programming Using Python
David I. Schneider